

Verifying ABA with Leslie

Gaspard Reghem

June 4, 2026

Abstract

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consist in building a simulation relation between the implementation and an implementation running in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex, thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions; making the construction easier.

1 Preliminaries

Probabilistic Labelled Transition System

Probabilistic forward Simulation

Equivalence in the Non-Probabilistic Setting

2 List of the Protocols

2.1 Main Protocol: Asynchronous Byzantine Agreement (ABA)

(taken from [ABDY22]) Processes start with a bit $\{0,1\}$ and may return a bit. a program implements ABA if it satisfies three properties: Validity, Agreement, Almost-Sure Termination.

- Validity: if a correct process returns b then a correct process had $1 - b$ as input.
- Agreement: if two correct processes return, then their output are equal.
- Termination: if $n - f$ correct processes take part to the protocol then, with probability 1, all correct processes will eventually return.

Algorithm 1 Ideal Implementation of ABA

Protocol 1. **Input:** $b \in \{0,1\}$

Code for every process:

send $\langle input, b \rangle$ to the third party

upon receiving $\langle output, b \rangle$ from third party
 return b

Code for third party:

$bind, call \leftarrow \perp, (\perp)^{Id}$

upon receiving $\langle input, b \rangle$ from a process id and $bind = \perp$:

$call(id) \leftarrow b$

upon $|F| + \{id \in Id \setminus F \mid call(id) = b\} \geq n - f$:

$bind, call \leftarrow b, (\perp)^{Id}$

if $\{id \in Id \setminus F \mid call(id) = 1 - b\} = 0$:

 send $\langle output, b \rangle$ to all

upon $call(id) = \perp$ and $bind \neq \perp$:

$call \leftarrow bind$ **or** μ

$\triangleright \mu$ can be any non-Dirac probability distribution over $\{0,1\}$

Definition 1. $ABA.Concrete := ABA.Impl[GBCA.Impl, WCC.Impl]$, the instantiation of $ABA.Impl$ with concrete implementations of the sub-protocols.

Definition 2. $ABA.Hybrid := ABA.Impl[GBCA.Spec, WCC.Spec]$, the instantiation of $ABA.Impl$ with ideal specifications of the sub-protocols.

Algorithm 2 Implementation of ABA taken from [ABDY22]

Protocol 2. **Input:** $b \in \{0, 1\}$

```

 $r \leftarrow 1$ 
 $decided \leftarrow \perp$ 
loop
   $(b, g) \leftarrow \text{GBCA}_r(b)$ 
   $c \leftarrow \text{WCC}_r$ 
  if  $b = \perp$  then
     $b \leftarrow c$ 
  else if  $g = A$  then
     $decided \leftarrow b$ 
  end if
   $r \leftarrow r + 1$ 
end loop

```

$\triangleright b \in \{0, 1, \perp\}, g \in \{A, B, C\}$
 $\triangleright$ common coin for round r
 $\triangleright$ keep participating to help others
 $\triangleright$ TODO: verify termination rule

2.2 ABA Sub-Protocols

Graded Binding Crusader Agreement (GBCA) (taken from [ABDY22, AW23]) Processes start with a binary input $\{0, 1\}$ and may return a value in $\{(0, A), (0, B), (\perp, C), (1, B), (1, A)\}$. A program implements GBCA if it satisfies four properties: Validity, Graded Agreement, Binding, Termination.

- Validity: if a correct process does not return (b, A) then a correct process had $1 - b$ as input.
- Graded Agreement: if two correct processes return (b, X) and (b', X') then $b = 0 \implies b' \neq 1$ and $X = A \implies X' \neq \perp$.
- Binding: when a correct process returns then a bound value b is defined and, in all extensions of the execution, correct processes do not return $(1 - b, B)$ or $(1 - b, A)$.
- Termination: if $n - f$ correct processes take part to the protocol then all correct processes will eventually return.

Protocol 3. Blablabla

Protocol 4. Parameters: n processes with at most $f < n/3$ corrupted. Each process p_i has input $b_i \in \{0, 1\}$.

Weak Common Coin (WCC) Processes start with no input and may return a bit. A program implements WCC if it satisfies three properties: ϵ -Goodness, Unpredictability, Termination.

- ϵ -Goodness: with probability ϵ , all correct processes return 0 (resp. 1).
- Unpredictability: Until $t + 1$ processes have called WCC, all outcomes are still possible.
- Termination: if $n - f$ correct processes take part to the protocol then all correct processes will eventually return.

Protocol 5. Blablabla

Algorithm 3 Ideal Implementation of GBCA

Input: $b \in \{0, 1\}$

Code for every process:

send $\langle input, b \rangle$ to the third party

upon receiving $\langle output, (b, g) \rangle$ from third party
return (b, g)

Code for third party:

$bind, grade, call \leftarrow \perp, \perp, (\perp)^{Id}$

upon receiving $\langle input, b \rangle$ from a process id :

$call(id) \leftarrow b$

upon $|F| + \{id \in Id \setminus F \mid call(id) = b\} \geq n - f$:

$bind \leftarrow b$

if $\{id \in Id \setminus F \mid call(id) = 1 - b\} = 0$:

$grade \leftarrow A$

else

$grade \leftarrow C$

upon $bind = \perp$ and $grade = A$ and not sent $\langle output, \cdot \rangle$ to process id :

send $\langle bind, B \rangle$ or $\langle output, (bind, A) \rangle$ to process id

upon $bind = \perp$ and $grade = C$ and not sent $\langle output, \cdot \rangle$ to process id :

send $\langle bind, B \rangle$ or $\langle output, (\perp, C) \rangle$ to process id

Algorithm 4 Implementation of GBCA taken from [ABDY22]

Input: $b \in \{0, 1\}$

send $\langle INPUT, b_i \rangle$ to all processes

upon receiving $\langle INPUT, b \rangle$ from at least $f + 1$ processes and not sent $\langle INPUT, b \rangle$ yet:
send $\langle INPUT, b \rangle$ to all

upon receiving $\langle INPUT, b \rangle$ from at least $n - f$ processes:

$Approved \leftarrow Approved \cup \{b\}$

if not sent $\langle ECHO, \cdot \rangle$ yet **then** send $\langle ECHO, b \rangle$ to all

wait until

(a) receiving $\langle ECHO, b \rangle$ from at least $n - f$ processes **or**

(b) receiving $\langle ECHO, \cdot \rangle$ from at least $n - f$ processes and $|Approved| > 1$

if (a) **then** send $\langle vote, b \rangle$ to all **else** send $\langle vote, \perp \rangle$ to all

wait until

(a) receiving $\langle VOTE, b \rangle$ from at least $n - f$ processes **or**

(b) receiving $\langle VOTE, \cdot \rangle$ from at least $n - f$ processes and $|Approved| > 1$

if (a) **then** send $\langle bind, b \rangle$ to all **else** send $\langle bind, \perp \rangle$ to all

wait until

(a) receiving $\langle VOTE, b \rangle$ from at least $n - f$ processes **or**

(b) receiving $\langle VOTE, \cdot \rangle$ from at least $n - f$ processes and there exists $b \in \{0, 1\}$ such that $\langle bind, b \rangle$ has been received once and $\langle vote, b \rangle$ has been received from $t + 1$ processes and $|Approved| > 1$ **or**

(c) receiving $\langle VOTE, \perp \rangle$ from at least $n - f$ processes and $|Approved| > 1$

if (a) **then** return (b, A) **elif** (b) **then** return (b, B) **else** return $(\perp, C)$

Algorithm 5 Ideal Implementation of WCC

Input: none

Code for every process:

send $\langle input, b \rangle$ to the third party

upon receiving $\langle output, b \rangle$ from third party
return b

Code for third party:

$r \leftarrow \mu$

$\triangleright \mu : 1 \mapsto \epsilon; 0 \mapsto \epsilon; \perp \mapsto 1 - 2\epsilon$

if $r \neq \perp$:

send $\langle output, r \rangle$ to all

else

for all $id \in Id$, send $\langle output, 0 \rangle$ **or** $\langle output, 1 \rangle$ to process id

Algorithm 6 Implementation of WCC (yet to be determined)

$x \leftarrow 0$

if $y > 0$ **then**

loop

end if

Protocol 6. Requires Gather and RSD

Definition 3. $WCC.Concrete := WCC.Impl[Gather.Concrete, RSD.Concrete]$, the instantiation of $WCC.Impl$ with fully concrete sub-protocols.

Definition 4. $WCC.Hybrid := WCC.Impl[Gather.Spec, RSD.Spec]$, the instantiation of $WCC.Impl$ with ideal specifications of the sub-protocols.

2.3 WCC Sub-Protocols

Gather (taken from [SA21, SA24, AFW25]) Processes start with an input $x \in X$ and may return a partial function $f : Proc \rightarrow X$. A program implements Gather if it satisfies four properties: Validity, Agreement, Common Core, Binding, Termination.

- Validity: if a correct process returns f and $f(q)$ is defined and q is correct then $f(q)$ is q 's input.
- Agreement: if two correct processes return f and g then $f(q)$ and $g(q)$ are equal or one of them is undefined.
- Binding Common Core: Once a correct process has returned, there exists $S \subseteq Proc$ of size at least $n - f$ such that all f returned by a correct process in the future is defined on S .
- Termination: if $n - f$ correct processes take part to Gather then all correct processes will eventually return.

Protocol 7. Blablabla

Protocol 8. Might require BRB

Definition 5. $Gather.Concrete := Gather.Impl[BRB.Impl]$, the instantiation of $Gather.Impl$ with the concrete implementation of BRB.

Algorithm 7 Ideal Implementation of Gather

Input: m

Code for every process:

send $\langle input, m \rangle$ to the third party

upon receiving $\langle output, f \rangle$ from third party:

return f

Code for third party:

$call, bind \leftarrow \emptyset, \emptyset$

upon receiving $\langle input, m \rangle$ from process id :

$call \leftarrow call \cup \{(id, m)\}$

if $|F \cup \{id \in Id \setminus F \mid call[id] \neq \perp\}| > n - f$ and $bind = \perp$:

$bind \leftarrow S \subseteq F \cup \{id \in Id \setminus F \mid (id, \cdot) \in call\}$ $\triangleright$ Non-deterministic affectation

upon $bind \neq \perp$:

for all $id \in Id$, select $bind \subseteq S \subseteq F \cup \{id \in Id \setminus F \mid (id, \cdot) \in call\}$ and

send $\langle output, \{(id, m) \in call \mid id \in S \setminus F\} \cup \{(id, \cdot) \mid id \in S \cap F\} \rangle$ to process id

Algorithm 8 Implementation of Gather from [AFW25]

Input: $b \in \{0, 1\}$

Sub-Protocols: $|Id|$ instances of BRB (process id is leader in BRB_{id})

$BRB_{id}.call(m)$

upon $BRB_i.return(m')$:

$AP \leftarrow AP \cup \{(i, m')\}$

upon $|AP| > n - f$ and not sent $\langle ECHO, \cdot \rangle$:

send $\langle echo, AP \rangle$ to all

upon receiving approved $\langle ECHO, AP_{id} \rangle$ from at least $n - f$ processes and not sent $\langle VOTE, \cdot \rangle$:

send $\langle VOTE, \bigcup AP_{id} \rangle$

upon receiving approved $\langle VOTE, AP_{id} \rangle$ from at least $n - f$ processes and not sent $\langle BIND, \cdot \rangle$:

send $\langle BIND, \bigcup AP_{id} \rangle$

upon receiving approved $\langle BIND, AP_{id} \rangle$ from at least $n - f$ processes:

return $\bigcup AP_{id}$

Definition 6. $\text{Gather.Hybrid} := \text{Gather.Impl}[\text{BRB.Spec}]$, the instantiation of Gather.Impl with the ideal specification of BRB.

Random Secret Draw (RSD) The RSD protocol is composed of two operations: Assignment and Retrieval. Assignment affects a random and secret value to each process and Retrieval provides these values to processes. Moreover, the API of RSD provides a notification function for each process that records which processes have a value assigned.

- Assignment Validity: if all correct processes call Assignment, then all correct processes will eventually receive a notification for each correct process.
- Assignment Totality: if a correct process receives a notification for process id then all correct processes will get such a notification.
- Randomness: If no correct process has called Assignment then the value returned by Retrieval for any process follows the uniform distribution.
- Secrecy: If no correct process has called Retrieval then the adversary has no information about the return value of Retrieval.
- Retrieval Termination: if all correct processes call Retrieval then the operation terminates for all correct processes

Protocol 9. Blablabla

Algorithm 9 Ideal Implementation of RSD

```

 $x \leftarrow 0$ 
if  $y > 0$  then
  loop
end if

```

Protocol 10. Require AVSS and BRB

Definition 7. $\text{RSD.Concrete} := \text{RSD.Impl}[\text{BRB.Impl}, \text{AVSS.Spec}]$, the instantiation of RSD.Impl with the concrete implementation of BRB. AVSS is used as a black-box, so its specification is plugged in directly.

Definition 8. $\text{RSD.Hybrid} := \text{RSD.Impl}[\text{BRB.Spec}, \text{AVSS.Spec}]$, the instantiation of RSD.Impl with the ideal specifications of BRB and AVSS.

2.4 Basic Primitives

Byzantine Reliable Broadcast (BRB) (taken from [Bra87]) Each instance of BRB has a designated *leader* with an input message m and processes may return a received message. A program implements BRB if it satisfies three properties: Validity, Totality and Agreement.

- Validity: if the leader is not corrupted then all correct processes must return its input.
- Agreement: if two correct processes return then they return the same value.
- Totality: if a correct process returns then all correct processes eventually return.

Protocol 11. Blablabla

Protocol 12. Requires no sub-protocol. Parameters: n processes with at most $f < n/3$ corrupted; leader ℓ with input m .

Algorithm 10 Implementation of RSD from [FKT22]

Input: $b \in \{0, 1\}$
Sub-Protocols: $|Id|$ instances of BRB (process id is leader in BRB_{id}) and $|Id|^2$ instances of AVSS (process id is dealer in $AVSS_{id}^k$)

Operation: $RSD.Sh()$
 $retrieveEnabled, S \leftarrow \perp, \emptyset$
for all $id' \in Id$, $x_{id'} \leftarrow \mu$ $\triangleright \mu$: Uniform distribution over the domain of secrets
 $AVSS_{id'}^{id'}.Sh.call(x_{id'})$
upon $AVSS_{id'}^{id'}.Sh.return()$:
 $S \leftarrow S \cup \{id'\}$
upon $|AP| > n - f$ and not called BRB_{id} :
 $BRB_{id}.call(S)$
upon $BRB_{id'}.return(S')$ and $\forall k \in S', AVSS_k^{id'}.Sh.return()$:
 $src[id'] \leftarrow S'$
event $ValueAssigned(id')$
 $\forall k \in S', AVSS_k^{id'}.Rec.call()$

Operation: $RSD.Rec()$
return $id' \mapsto \sum_{k \in src[id']} AVSS_k^{id'}.Rec.return()$

Algorithm 11 Ideal Implementation of BRB

Input: $m \in \{0, 1\}^*$ for the leader

Code for the leader:
send $\langle input, m \rangle$ to the third party

Code for every process:
upon receiving $\langle output, m \rangle$ from third party:
return m

Code for third party:
upon receiving $\langle input, m \rangle$ from leader:
if leader is correct:
send $\langle output, m \rangle$ to all
else
send $\langle output, m' \rangle$ to all $\triangleright$ Arbitrary m'

Algorithm 12 Implementation of BRB taken from [Bra87]

Code for the leader ℓ with input m :

send $\langle \text{INIT}, m \rangle$ to all processes

Code for every process p_i :

upon first reception of $\langle \text{INIT}, m \rangle$ from ℓ **do**

send $\langle \text{ECHO}, m \rangle$ to all processes

upon reception of $\langle \text{ECHO}, m \rangle$ from at least $n - f$ distinct processes **do**

if $\langle \text{READY}, \cdot \rangle$ has not yet been sent **then**

send $\langle \text{READY}, m \rangle$ to all processes

upon reception of $\langle \text{READY}, m \rangle$ from at least $f + 1$ distinct processes **do**

if $\langle \text{READY}, \cdot \rangle$ has not yet been sent **then**

send $\langle \text{READY}, m \rangle$ to all processes

upon reception of $\langle \text{READY}, m \rangle$ from at least $2f + 1$ distinct processes **do**

deliver m

Asynchronous Verifiable Secret Sharing (AVSS) AVSS is actually a pair of protocol (Sh, Ret) with a designated *dealer* owning a secret s . A program implements a $(1 - \epsilon)$ -correct AVSS if it satisfies six properties: Valid Termination, Total Termination, *Rec* Termination, Binding, Secrecy.

- **Validity:** if the dealer is correct and a correct process returns from *Ret* then the return value is the dealer's secret.
- **Sharing Termination:** if the dealer is correct and calls *Sh* then all correct processes will eventually return from *Sh*.
- **Sharing Totality:** if a correct process returns from *Sh* then all correct processes will eventually return from *Sh*.
- **Retrieval Termination:** if all correct processes call *Ret* then all correct processes will eventually return from *Ret*.
- **Commitment:** if a correct process returns from *Sh* then there exists a secret s such that correct process can only retrieve s from that point.
- **Secrecy:** if the dealer is correct and no correct process has called *Ret* then the adversary has no information about the future retrieved value.

Protocol 13. Blablabla

Algorithm 13 AVSS Specification

$x \leftarrow 0$

if $y > 0$ **then**

loop

end if

AVSS is used as a blackbox so no implementation.

2.5 Summary of Properties

The linear safety properties are preserved by Segala’s probabilistic forward simulation. The branching safety properties can be transformed into linear ones by enhancing the interface with an external binding label. Segala’s simulation does not preserve liveness by default. However, it can be strengthened following [DSW22] to preserve liveness in a complete manner.

	Linear Safety	Branching Safety	Liveness
ABA	Validity & Agreement	Binding	Termination
GBCA	Validity & Agreement		Termination
WCC	Validity & Agreement		Termination
Gather		Binding Common Core	Assignment Validity & Assignment Totality & Retrieval Termination
RSD		Commitment	Totality
BRB	Sharing Termination & Sharing Totality & Retrieval Termination		
AVSS	Validity		

Table 1: Table of CTL Properties

Segala’s simulation was designed to be sound and complete for trace distribution so it preserves probabilistic safety properties. Note that Unpredictability is a probabilistic safety property. For probabilistic liveness, we will rely on the liveness preservation as it is sound (but not complete). The lack of completeness is not an issue because the limitations of the termination proof rule is probabilistic convergence which is never dealt with by our simulations. The probabilistic convergence argument will only be necessary to prove that the ideal implementation of ABA almost-surely terminates and can be discharged to the super-Martingale proof rule of [EMMS26]. For Secrecy, we use the same trick as for branching: enhancing the interface.

	Probabilistic Safety	Probabilistic Liveness	Secrecy
ABA	ϵ -Goodness & Unpredictability	Almost-Sure Termination	
GBCA			
WCC			
Gather	Randomness		Secrecy
RSD			
BRB			
AVSS			Secrecy

Table 2: Table of non-CTL Properties

3 Proof Decomposition

The end goal is the correctness of `ABA.impl`. We establish it via a simulation by `ABA.Spec` and transport the spec’s correctness. The simulation itself is decomposed recursively along the sub-protocol structure.

For a protocol P with concrete sub-protocols $Q_1, \dots, Q_k$, the step “ $P.Spec$ simulates $P.impl$ ” is split into:

1. (Substitution) $P.\text{Impl}[Q_1.\text{Spec}, \dots, Q_k.\text{Spec}]$ is simulated by $P.\text{Impl}$, using the child simulations $Q_i.\text{Spec}$ simulates $Q_i.\text{Impl}$ and compositionality.
2. (Core) $P.\text{Spec}$ simulates the hybrid $P.\text{Impl}[Q_1.\text{Spec}, \dots, Q_k.\text{Spec}]$.
3. (Transitivity) Compose (1) and (2).

Leaves of the recursion (no sub-protocol, or only the black-box AVSS.Spec) skip the substitution step.

3.1 Top-level

Theorem 9. ABA.Concrete *satisfies Validity, Agreement and Almost-Sure Termination.*

Proof. Transport the corresponding properties of ABA.Spec along the simulation 10. □

3.2 ABA

Theorem 10. ABA.Spec *simulates* ABA.Concrete .

Proof. Transitivity of simulation applied to 11 and 12. □

Lemma 11. ABA.Hybrid *is simulated by* ABA.Concrete .

Proof. Compositionality of simulation applied to 16 and 13. □

Lemma 12. ABA.Spec *simulates* ABA.Hybrid .

3.3 WCC

Theorem 13. WCC.Spec *simulates* WCC.Concrete .

Proof. Transitivity of simulation applied to 14 and 15. □

Lemma 14. WCC.Hybrid *is simulated by* WCC.Concrete .

Proof. Compositionality of simulation applied to 17 and 20. □

Lemma 15. WCC.Spec *simulates* WCC.Hybrid .

3.4 GBCA

Theorem 16. GBCA.Spec *simulates* GBCA.Impl . *No sub-protocol: direct proof.*

3.5 Gather

Theorem 17. Gather.Spec *simulates* Gather.Concrete .

Proof. Transitivity of simulation applied to 18 and 19. □

Lemma 18. Gather.Hybrid *is simulated by* Gather.Concrete .

Proof. Compositionality of simulation applied to 23. □

Lemma 19. Gather.Spec *simulates* Gather.Hybrid .

3.6 RSD

Theorem 20. RSD.Spec *simulates* RSD.Concrete .

Proof. Transitivity of simulation applied to [21](#) and [22](#). □

Lemma 21. RSD.Hybrid *is simulated by* RSD.Concrete . *Only BRB is substituted; AVSS.Spec stays fixed on both sides.*

Proof. Compositionality of simulation applied to [23](#). □

Lemma 22. RSD.Spec *simulates* RSD.Hybrid .

3.7 BRB

Theorem 23. BRB.Spec *simulates* BRB.Impl . *No sub-protocol: direct proof.*

Bibliography

- [ABDY22] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In *41st ACM Symposium on Principles of Distributed Computing*, pages 381–391, 2022.
- [AFW25] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond canonical rounds: Communication abstractions for optimal byzantine resilience. *CoRR*, abs/2510.04310, 2025.
- [AW23] Hagit Attiya and Jennifer L. Welch. Multi-valued connected consensus: A new perspective on crusader agreement and adopt-commit. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, 2023.
- [Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [DSW22] Brijesh Dongol, Gerhard Schellhorn, and Heike Wehrheim. Weak Progressive Forward Simulation Is Necessary and Sufficient for Strong Observational Refinement. In Bartek Klin, Slawomir Lasota, and Anca Muscholl, editors, *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:23, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [EMMS26] Constantin Enea, Rupak Majumdar, Harshit Jitendra Motwani, and V. R. Sathiyarayanan. Verifying almost-sure termination for randomized distributed algorithms. *Proc. ACM Program. Lang.*, 10(POPL):1412–1441, 2026.
- [FKT22] Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. Distributed randomness from approximate agreement, 2022.
- [SA21] Gilad Stern and Ittai Abraham. Living with asynchrony: the gather protocol. <https://decentralizedthoughts.github.io/2021-03-26-living-with-asynchrony-the-gather-protocol>, 2021.
- [SA24] Gilad Stern and Ittai Abraham. Gather with binding and verifiability. <https://decentralizedthoughts.github.io/2024-01-09-gather-with-binding-and-verifiability/>, 2024.